COIMBATORE INSTITUTE OF TECHNOLOGY—641014
MSC – SOFTWARE SYSTEMS

PROJECT RECORD
VISUAL DSA — INTERACTIVE DATA STRUCTURES AND ALGORITHMS VISUALIZATION PLATFORM

(Interactive Learning, Algorithm Tracing & AI Tutoring)
A Full-Stack React & Python Web Application

NAME	**DHARANEESH V**
REGISTER NUMBER	**2503717672621011**
COURSE	**M.Sc.SS**
SUBJECT CODE	**25SS322**
SUBJECT	**PYTHON PROGRAMMING LABORATORY**
SEMESTER	**III**
ACADEMIC YEAR	**2026 – 2027**

SYNOPSIS

Title: Visual DSA — Interactive Data Structures and Algorithms Visualization Platform

Background: Data structures and algorithms (DSA) are the foundation of computer science education and software engineering proficiency. However, abstract concepts such as dynamic pointer manipulation, recursive call-stack unwinding, tree balancing, and algorithmic step execution are notoriously difficult to master using static textbook diagrams and dry code snippets. Learners frequently struggle to mentalise dynamic memory states and time complexity transitions, resulting in high cognitive friction and retention challenges.

Proposed System: Visual DSA is a full-stack educational web application engineered with React 19, TypeScript, Node.js/Express, MongoDB, and a Python FastAPI microservice integrated with local Ollama LLMs. The platform digitizes the algorithm learning lifecycle through synchronized step-by-step canvas animations, real-time pseudocode line execution, live variable inspectors, interactive auto-graded quizzes, persistent mastery tracking, and an on-demand AI tutor (Lemmy Assistant) for natural language algorithmic guidance.

Key Features:

- Interactive visualizers for Arrays, Linked Lists, Stacks, Queues, Circular Queues, Searching, and Sorting with step-by-step playback controls, speed adjustment, and custom input datasets.
- Synchronized pseudocode execution panel that dynamically highlights active code lines alongside asymptotic complexity breakdowns (Best, Average, Worst Time and Space).
- Intelligent AI Tutor (Lemmy Assistant) powered by a Python microservice with local LLMs (Ollama) providing code explanations, complexity analysis, and debugging tips.
- Self-assessment practice suite featuring topic-specific multiple-choice quizzes, real-time scoring, answer explanations, and automated progress recording.
- Persistent learner progress tracking with overall completion metrics, concept mastery meters, and cryptographically verifiable PDF Certificates of Completion.

Technologies Used: React 19 with Vite, TypeScript, and Tailwind CSS (frontend); Node.js and Express.js (REST API backend); MongoDB with Mongoose ODM (database); Python 3.10+, FastAPI, and Ollama LLM (AI service); JSON Web Tokens (JWT), bcryptjs, Canvas Confetti, and jsPDF.

Expected Outcome: An intuitive, pedagogically robust, and responsive web platform that bridges theoretical algorithmic principles with dynamic graphical execution, enabling students to master complex computational paradigms with confidence and clarity.

OBJECTIVE

The primary objective of this project is to design and develop an interactive, full-stack visualization and self-paced learning platform that demystifies data structures and algorithms through intuitive animations and intelligent tutoring. The specific objectives are:

1. To develop interactive, graphical animations for fundamental data structures (Arrays, Singly Linked Lists, Stacks, Queues, Circular Queues) and core algorithms (Linear Search, Binary Search, Bubble, Selection, Insertion, Quick, and Merge Sort).

2. To provide granular, bidirectional execution playback controls (Play, Pause, Step Forward, Step Backward, Reset, and Speed Slider) for detailed inspection of state transitions.

3. To synchronize visual canvas transformations with real-time highlighted pseudocode lines and active variable states.

4. To build an auto-graded practice assessment engine with randomized question pools, immediate feedback, and conceptual explanations.

5. To engineer an AI-powered tutoring service using Python FastAPI and local Ollama LLMs for contextual explanations and algorithmic query resolution.

6. To implement secure user authentication, bcrypt password hashing, and session persistence using JSON Web Tokens (JWT).

7. To record learner progress, calculate topic mastery percentages, log activity sessions, and issue verifiable PDF Certificates of Completion.

SCOPE

The system is designed for computer science students, engineering faculty, coding bootcamp participants, and technical interview candidates seeking an intuitive, visual approach to mastering data structures and algorithms.

Scope of the Project

- Interactive Data Structure Visualizers: Graphical representation of arrays with element shifting; linked lists with pointer redirection; stacks with LIFO push/pop; and linear/circular queues with wraparound buffer tracking.
- Algorithm Animation Playground: Real-time execution of searching and sorting algorithms with color-coded comparison, swap, and sorted boundary highlights.
- Synchronized Theory & Pseudocode: Collapsible instruction panels detailing formal definitions, asymptotic complexities, and line-by-line algorithm code.
- Practice & Auto-Graded Assessments: Multiple-choice evaluations across all topics with instant score computation, performance statistics, and review summaries.
- AI-Assisted Tutoring: Context-aware assistance via Lemmy AI, delivering code hints and time complexity breakdowns through a Python FastAPI microservice.
- Learner Analytics & Certification: Persistent tracking of completed topics, operations count, study time, and client-side PDF certificate generation.

Limitations

- The application operates as a modern web application requiring an active Node.js server and MongoDB connection.

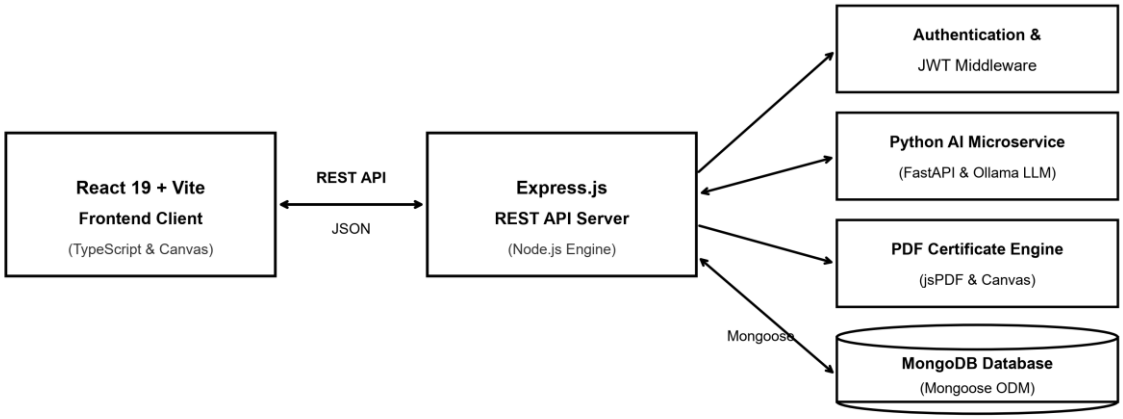
- AI Tutoring depends on an active local Ollama runtime; if the Python service is offline, fallback guidance is displayed.
- Real-time collaborative whiteboarding and multi-user synchronized classrooms are outside the present implementation scope.
- Advanced graph and dynamic programming visualizers (Dijkstra, Bellman-Ford, Floyd-Warshall) are planned for subsequent versions.

PROJECT DESCRIPTION

1. Overview

Visual DSA is a multi-tier educational platform engineered to eliminate cognitive barriers in algorithmic learning. The React 19 frontend communicates with an Express.js REST API for authentication, user profiles, and progress metrics stored in MongoDB via Mongoose ODM. An auxiliary Python FastAPI microservice connects the frontend to a local Ollama LLM runtime, delivering low-latency, private, and contextual tutoring assistance.

2. System Architecture



System architecture of the Visual DSA Platform

3. Modules

Module	Description
Array Visualizer	Dynamic contiguous memory manipulation (Insert, Delete, Search, Update) with animated element shifting and index tracking.
Linked List Visualizer	Singly linked list node insertion (Head, Tail, Index), deletion, and reversal with animated pointer redirection and null termination.
Stack Visualizer	LIFO stack operations (Push, Pop, Peek, Clear) with top pointer tracking and overflow/underflow boundary protection.
Queue & Circular Queue	FIFO linear queue and circular buffer visualizers with dynamic front/rear pointer tracking and modulo wraparound logic.
Searching Algorithms	Step-by-step execution of Linear Search and Binary Search with low, mid, and high pointer indicators and comparison counts.
Sorting Algorithms	Interactive sorting playground for Bubble, Selection, Insertion, Quick, and Merge sort with color-coded comparison and swap states.

Module	Description
Practice & Quiz Engine	Topic-based multiple-choice assessment suite featuring auto-grading, randomized question pools, and detailed review feedback.
Progress & Dashboard	Real-time dashboard displaying curriculum completion percentage, concept mastery meters, active study time, and activity history.
AI Tutor (Lemmy Assistant)	FastAPI and Ollama LLM integration offering contextual algorithmic hints, code debugging, and theoretical explanations.
Certificate Generator	Automated client-side PDF certificate creation with learner name, completion timestamp, and verification hash.
User Authentication	Secure account registration, JWT session verification, bcrypt password hashing, and profile personalization.

4. Technology Stack

Layer	Technologies
Frontend	React 19 (Vite), TypeScript, Tailwind CSS, Lucide icons, HTML5 Canvas, Canvas Confetti
Backend API	Node.js, Express.js, cors, dotenv
Database	MongoDB with Mongoose ODM 8.x
Security	JSON Web Tokens (jsonwebtoken), bcryptjs
AI Microservice	Python 3.10+, FastAPI, Uvicorn, httpx, Ollama LLM (Llama 3 / Mistral)
PDF & Export	jsPDF, html2canvas (Certificates), python-docx, docx2pdf

5. Software Requirements

- Node.js v18.x or higher with npm package manager.
- MongoDB Community Server (local) or MongoDB Atlas connection URI.
- Python 3.10+ runtime with FastAPI and Uvicorn.
- Ollama local AI runtime with llama3 or mistral pulled.
- Modern web browser (Google Chrome, Microsoft Edge, or Firefox).

6. Working of the System

1. The learner registers or logs in; the Express API verifies credentials using bcrypt and issues a signed JWT.

2. The learner selects a data structure or algorithm module; frontend initializes data structures and pre-computes steps.

3. The user triggers playback controls; the engine animates memory states synchronized with highlighted pseudocode lines.

4. The learner completes practice quizzes; the client scores answers and submits results to the MongoDB progress collection.

5. The learner queries Lemmy AI assistant; prompts are routed to the Python FastAPI microservice interfacing with Ollama.

6. Upon completing curriculum topics, the learner generates and downloads a cryptographically verifiable PDF Certificate.

DATABASE DESIGN

The platform uses MongoDB, a document database (database: visual_dsa). Data is structured into 6 schema-validated collections managed through Mongoose ODM 8.x. Compound indexes ensure rapid progress queries, quiz response logging, and user session verification.

1. users Collection

Field Name	Data Type	Description / Constraint
_id	ObjectId	Primary Key
username	String	Unique login handle (Indexed, required)
email	String	Unique user email address (Indexed, lowercase)
password	String	bcrypt-hashed password (10 salt rounds)
role	String	Access role: 'student' or 'admin' (Default: 'student')
tokenVersion	Number	Session counter for token revocation
failedAttempts	Number	Failed login attempts counter
lockedUntil	Date	Account temporary lockout timestamp
createdAt	Date	Account creation ISO timestamp

2. progresses Collection

Field Name	Data Type	Description / Constraint
_id	ObjectId	Primary Key
userId	ObjectId	Foreign Key -> users._id (Indexed)
topic	String	DSA module topic key ('array', 'sorting')
concept	String	Specific algorithm name ('binary_search')
completed	Boolean	Topic completion flag (Default: false)
completionPct	Number	Mastery percentage score (0 to 100)
timeSpent	Number	Total active learning duration in seconds
operationsCount	Number	Count of visualizer operations executed
lastAccessed	Date	Timestamp of most recent topic activity

Index: Compound Super key index on { userId: 1, topic: 1 }

3. practices Collection

Field Name	Data Type	Description / Constraint
_id	ObjectId	Primary Key
userId	ObjectId	Foreign Key -> users._id (Indexed)
topicId	String	Target module identifier ('arrays', 'stack')
score	Number	Total marks scored in quiz attempt
totalQuestions	Number	Total number of questions in quiz
accuracy	Number	Percentage accuracy scored (0 to 100)
responses[]	Array	Array of selected option indices
attemptedAt	Date	Quiz submission ISO timestamp

Index: Compound Super key index on { userId: 1, topicId: 1, attemptedAt: -1 }

4. activities Collection

Field Name	Data Type	Description / Constraint
_id	ObjectId	Primary Key
userId	ObjectId	Foreign Key -> users._id (Indexed)
action	String	Action: 'LOGIN', 'VISUALIZE', 'QUIZ_SUBMIT'
module	String	Associated topic key ('array', 'sorting', 'ai_tutor')
details	String	Descriptive operation details
durationSeconds	Number	Session duration spent on action
timestamp	Date	Event occurrence ISO timestamp

Index: Compound index on { userId: 1, timestamp: -1 }

5. topics Collection

Field Name	Data Type	Description / Constraint
_id	ObjectId	Primary Key
topicId	String	Unique topic slug ('stack', 'bubble_sort')
title	String	Full module title ('Bubble Sort Visualizer')
category	String	Classification: 'Data Structures' or 'Algorithms'
timeComplexity	String	Asymptotic bounds (Best, Average, Worst)
spaceComplexity	String	Auxiliary memory complexity (e.g. O(1), O(n))
pseudocode	Array	Line-by-line algorithm implementation statements
orderIndex	Number	Display sorting order index

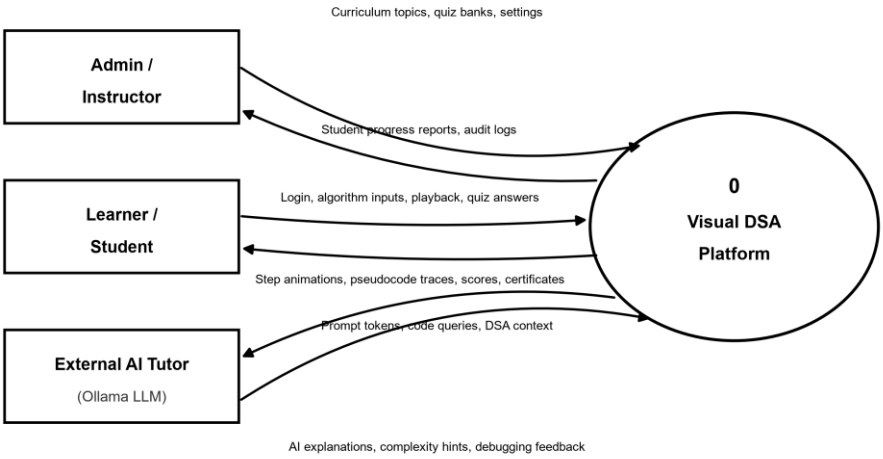
6. certificates and settings Collections (Configuration)

Field Name	Data Type	Description / Constraint
_id	ObjectId	Primary Key
userId	ObjectId	Foreign Key -> users._id (Indexed)
certificateNumber	String	Unique certificate identifier ('VDSA-2026-X892')
issueDate	Date	Curriculum completion timestamp
grade	String	Performance grade ('Distinction', 'Pass')
verificationHash	String	SHA-256 cryptographic credential verification hash
appVersion	String	Platform release version ('v2.4.0')
aiModelEndpoint	String	FastAPI microservice endpoint URI
quizPassThreshold	Number	Minimum passing score percentage (70%)

DATA FLOW DIAGRAM (DFD)

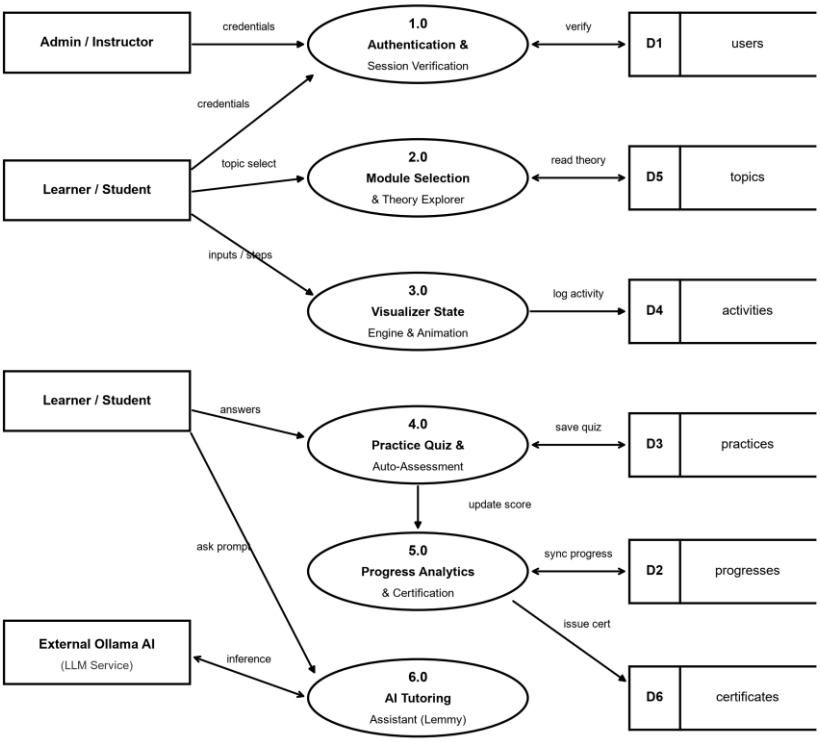
Level 0 – Context Diagram

The context diagram shows the whole system as one process exchanging data with three external entities: the Administrator / Instructor, the Learner / Student, and the External AI Service (Ollama LLM).



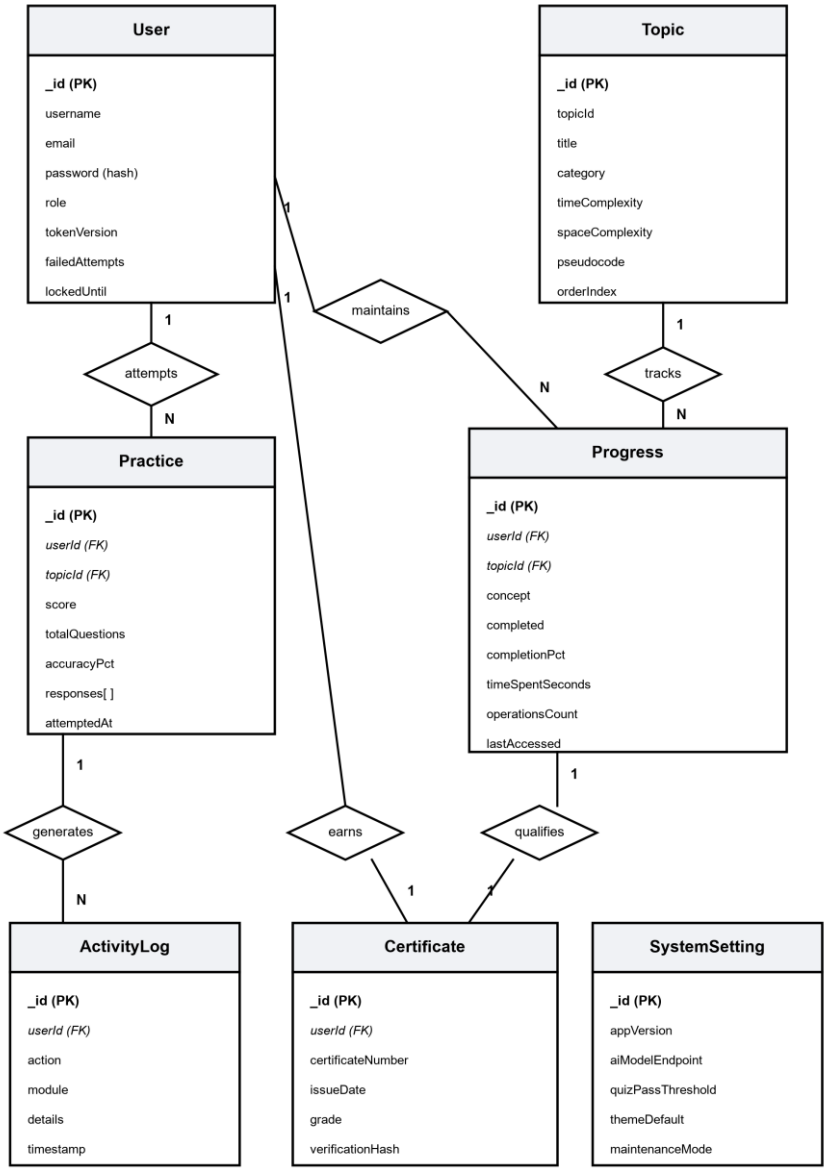
Level 1 DFD

Level 1 splits the system into six processes: Authentication & Session Verification (1.0), Module Selection & Theory Explorer (2.0), Visualizer State Engine (3.0), Practice Quiz & Auto-Assessment (4.0), Progress Analytics & Certification (5.0), and AI Tutoring Assistant (6.0), coordinated with six persistent data stores (D1–D6).



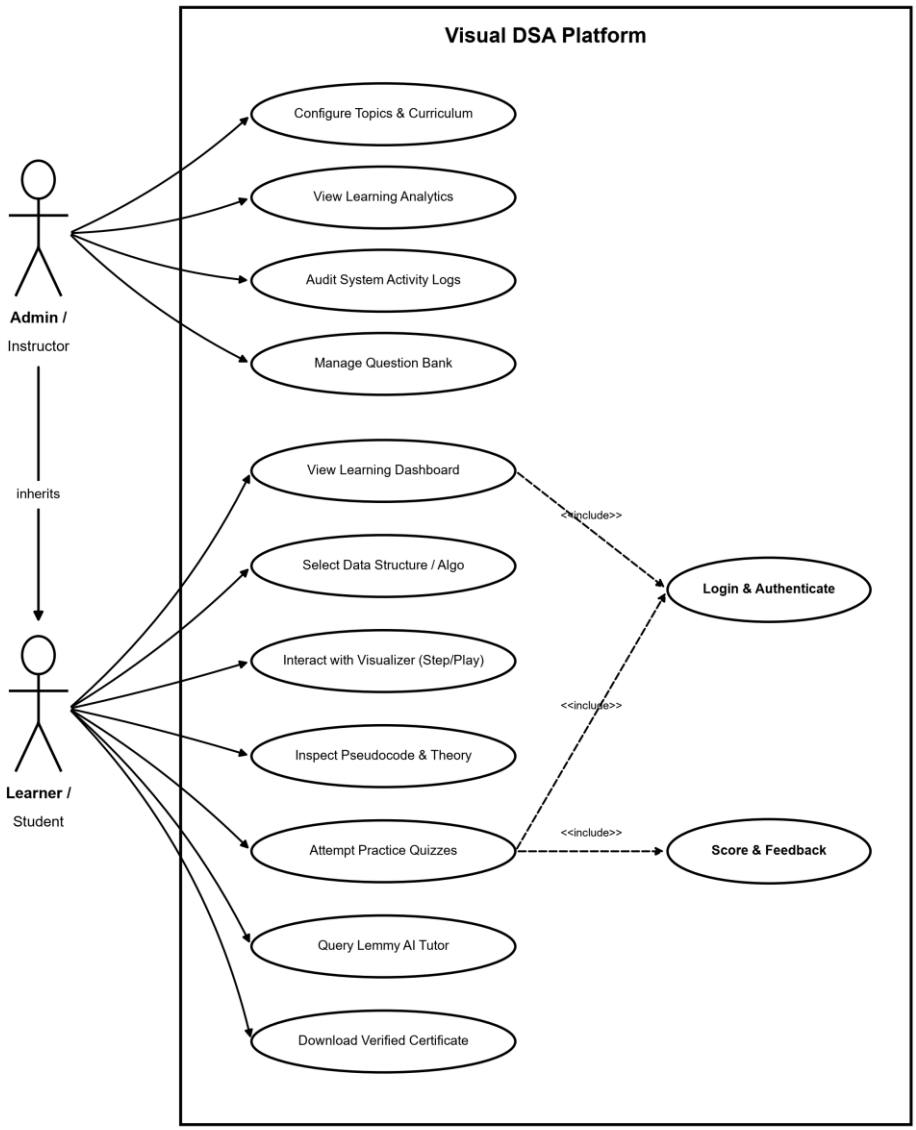
ER DIAGRAM

The entity-relationship diagram shows the collections and their relationships. One User maintains many Progress records (1 : N), one User attempts many Practice quizzes (1 : N), one User generates many ActivityLog entries (1 : N), and one User earns one Certificate (1 : 1). Topic tracks progress and evaluates quizzes, while SystemSetting provides platform configuration.



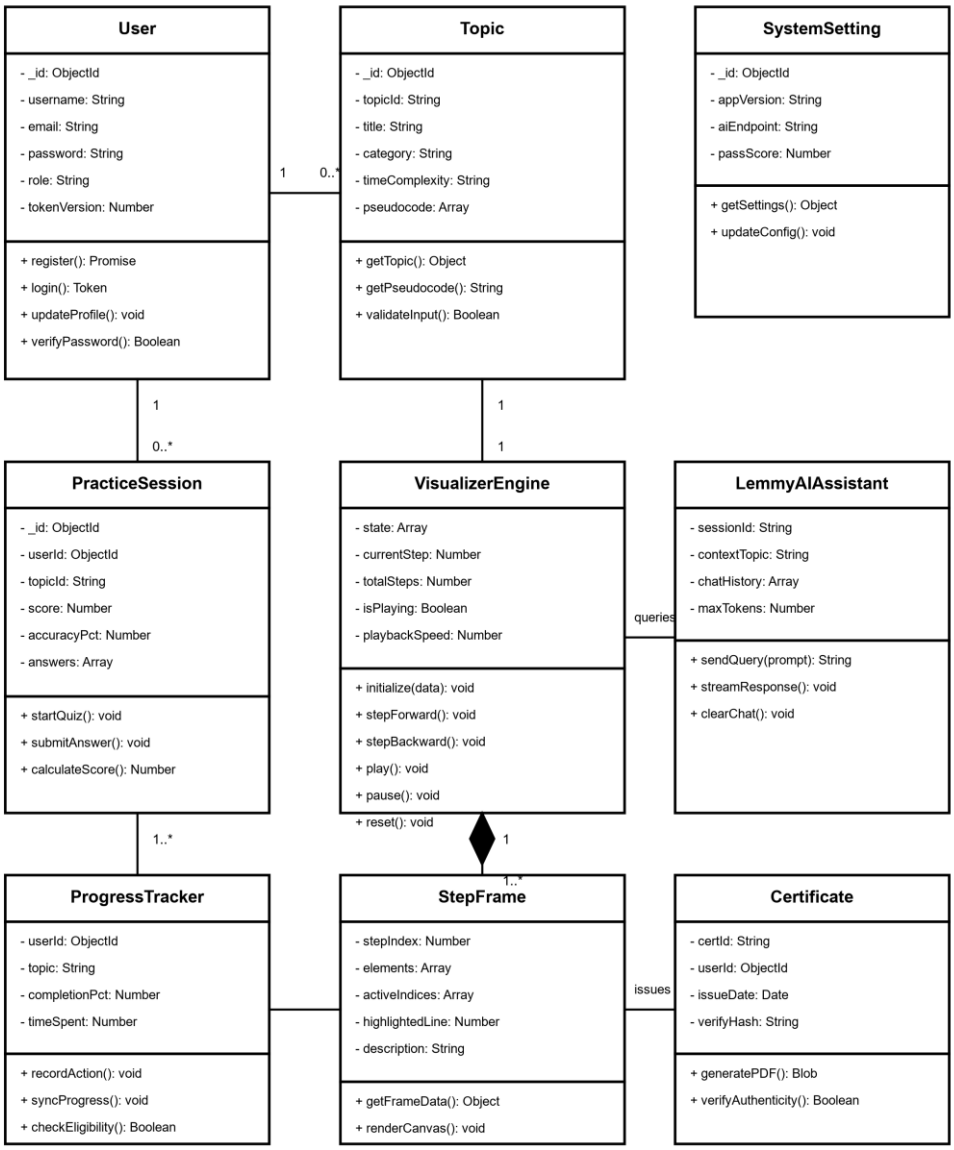
USE CASE DIAGRAM

Two actors interact with the system. The Learner / Student explores data structures, executes algorithm animations, attempts practice quizzes, queries the AI tutor, and downloads certificates. The Admin / Instructor inherits learner functionality and additionally manages topics, analytics, activity logs, and question banks.



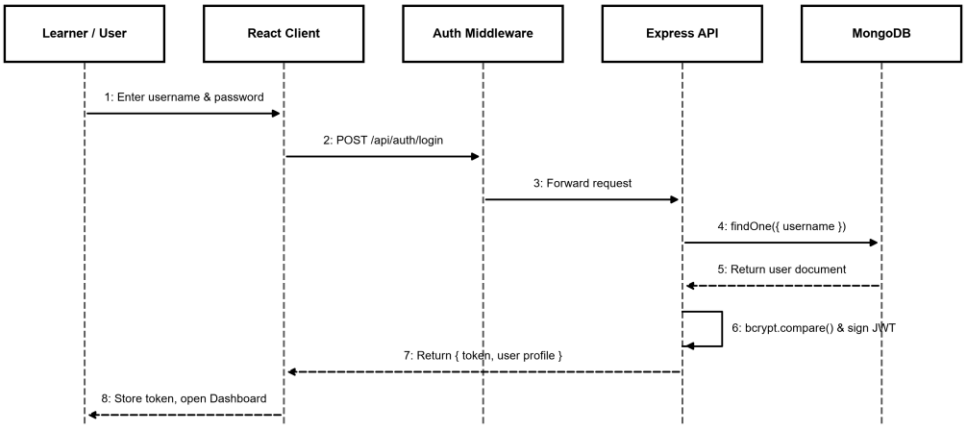
CLASS DIAGRAM

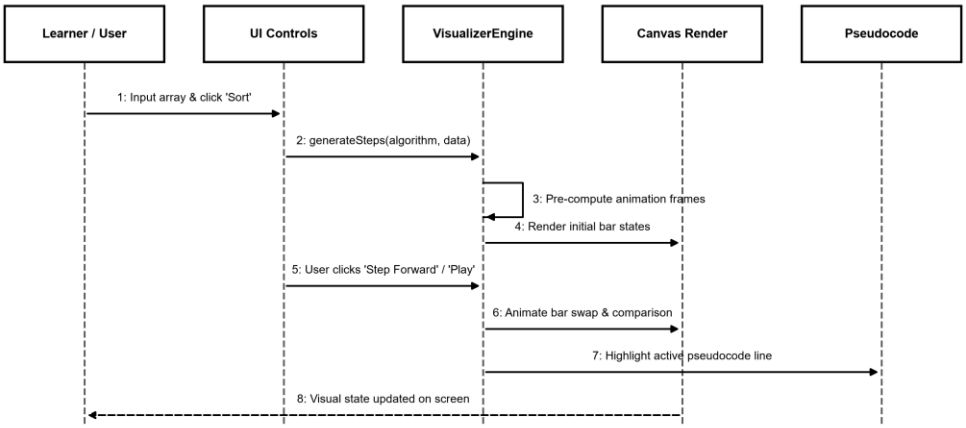
The class diagram shows the core classes with attributes and methods. VisualizerEngine aggregates StepFrame instances via composition to manage animation state frames. User coordinates with ProgressTracker and PracticeSession, while LemmyAIAssistant manages conversational prompts with the AI microservice.

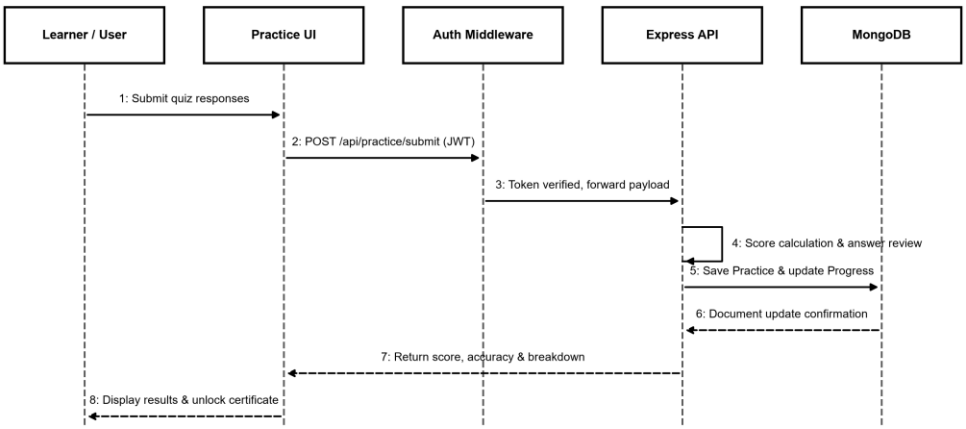


SEQUENCE DIAGRAM

The sequence diagrams illustrate object interactions across three critical workflows: User login and session verification; algorithm visualization with pseudocode synchronization; and quiz assessment with persistent progress synchronization.







SCREENSHOTS (FORMS)

The following screenshots present key interfaces of the Visual DSA application: the Home Learning Dashboard, the Interactive Algorithm Visualizer with live tracing, the Self-Assessment Practice Quiz, and the Verifiable Certificate of Completion.

Fig 1. Home Dashboard & Topic Catalog

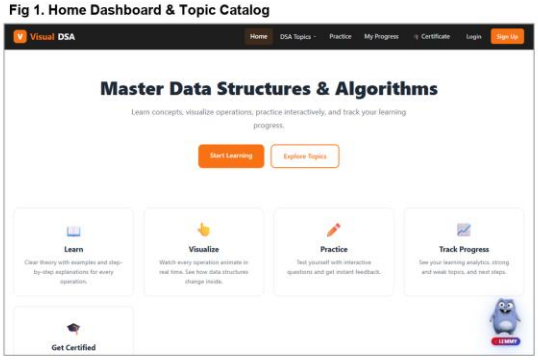


Fig 2. Interactive Algorithm Visualizer & Tracing

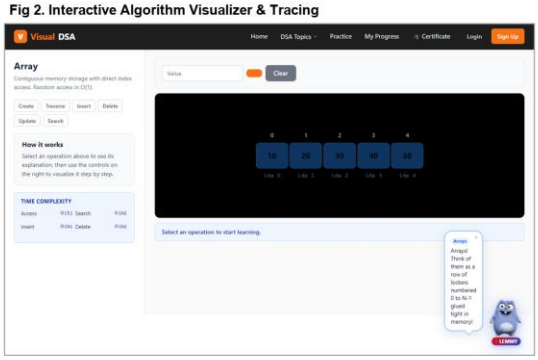


Fig 3. Practice Quiz & Auto-Graded Assessment

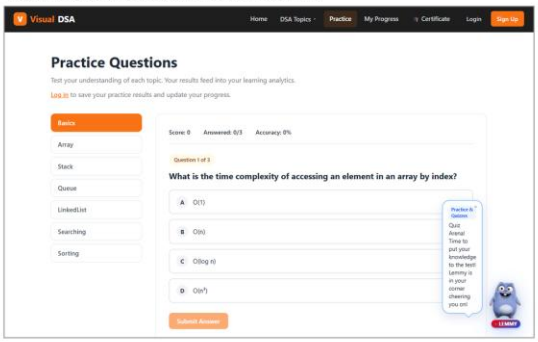
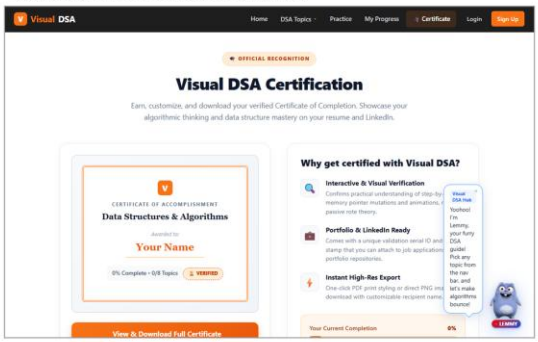


Fig 4. Verifiable PDF Certificate of Completion



TEST REPORT

The following test cases were prepared to verify the functional operations, visualization fidelity, assessment logic, and AI tutor responsiveness of Visual DSA.

TC ID	Module	Test Scenario	Expected Result	Status
TC01	Authentication	Login with valid credentials (student / pass123)	Authentication successful, JWT stored, opens Dashboard	Pass
TC02	Authentication	Login with invalid password	HTTP 401 returned; access denied with error banner	Pass
TC03	Access Control	Unauthorized access to protected route	Request intercepted; redirect to Login modal	Pass
TC04	Array Visualizer	Insert value at index 2	Array shifts elements right; visual animation executes	Pass
TC05	Linked List	Delete node at tail	Pointer updates, node fades out; length decremented	Pass
TC06	Stack Visualizer	Push element when stack is full	Overflow alert shown; element rejected cleanly	Pass
TC07	Circular Queue	Enqueue beyond buffer capacity	Rear pointer wraps around to index 0 with alert	Pass
TC08	Binary Search	Search target in sorted array	Low, mid, high pointers update step by step	Pass
TC09	Sorting Engine	Execute Bubble Sort on random array	Comparative swaps animate; active pseudocode highlights	Pass
TC10	Practice Quiz	Submit quiz with 3 correct answers	Score calculated as 100%; saved to database	Pass
TC11	AI Tutor (Lemmy)	Query time complexity of Merge Sort	Context sent to FastAPI; Ollama streams O(n log n)	Pass
TC12	Progress Tracker	Complete all topics in module	Overall curriculum completion % updates in real time	Pass
TC13	Certificate Engine	Generate certificate on completion	High-resolution PDF created with unique verification ID	Pass

CONCLUSION AND FURTHER ENHANCEMENTS

Conclusion

The Visual DSA (Interactive Data Structures and Algorithms Visualization Platform) successfully replaces passive textbook learning with an automated, robust, and interactive full-stack architecture. By providing a centralized digital environment for algorithm animations, synchronized pseudocode execution, dynamic variable tracking across execution states, auto-graded quizzes, verifiable PDF completion certificates,

and conversational AI tutoring powered by local Ollama LLMs, the platform eliminates conceptual ambiguity and learning friction. It provides computer science students and educators with transparent, real-time insight into internal data structure mutations and asymptotic complexity analysis.

Further Enhancements

- Advanced Graph & Dynamic Programming Visualizers: Direct integration of graph visualizers for Dijkstra's shortest path, Bellman-Ford, Prim's and Kruskal's minimum spanning trees, and dynamic programming matrices (0/1 Knapsack, Longest Common Subsequence).
- Collaborative Multi-User Classrooms: WebSocket-assisted multi-user synchronization allowing university instructors to broadcast live algorithm execution traces and pointer mutations across student screens simultaneously.
- In-Browser Code Execution Sandbox: WebAssembly-powered C++, Java, and Python code compilers enabling students to write custom algorithm implementations and watch them animate in real time.
- Voice-Activated Conversational AI Tutoring: Hardware-accelerated speech-to-text and voice synthesis integration with Lemmy AI for natural conversational questions and hands-free code debugging.
- Mobile Native Applications: Cross-platform iOS and Android mobile versions developed with React Native and Canvas acceleration for self-paced algorithm practice on mobile devices.
- Enterprise University LMS Integration: Centralized institutional portal with LTI (Learning Tools Interoperability) protocol support for automated gradebook synchronization with Moodle, Canvas, and Blackboard.